Ubuntu Boot Troubleshooting: Comprehensive Session Summary

A detailed recap of the investigation on a 2019 Intel MacBook Pro with Apple T2 hardware, followed by an intuitive explanation of Linux booting and why the final GDM workaround succeeded.

Part 1: The Problem We Were Trying to Solve

1. Hardware and operating system

Computer: 2019 MacBook Pro (MacBookPro16,1)

Operating system: Ubuntu Linux

Kernel: T2 Linux kernel

Integrated GPU: Intel UHD Graphics 630 (i915)

Dedicated GPU: AMD Radeon Pro 5500M (amdgpu)

Desktop environment: GNOME

Display manager: GDM

Graphical session: Wayland

Bootloader: GRUB

Init system: systemd

The Apple T2 platform and dual-GPU graphics stack make this installation more complicated than a typical Ubuntu laptop.

2. Original symptoms

On an intermittent basis, the computer showed the Ubuntu loading animation, briefly froze, and then displayed a black screen that remained backlit. No graphical login appeared. Occasionally, attempts to access GRUB for troubleshooting landed at the grub> prompt instead of the usual menu. These symptoms did not necessarily have the same cause.

Part 2: Our Troubleshooting Investigation

3. First hypothesis: a kernel regression

We tried an older T2 Linux kernel, 7.2.6-1-t2-noble, and removed a newer version. The older kernel initially booted, but the black-screen problem later returned. This weakened the theory that the newer kernel alone caused the failure. We therefore examined the transition from kernel startup to the graphical desktop.

4. Investigating GRUB

We inspected the GRUB command prompt and its partition listings, looked for /boot/grub/grub.cfg, and used normal to resume the boot sequence. GRUB was able to find and launch Ubuntu, which subsequently reached the splash screen and sometimes the desktop. The user clarified that GRUB was normally invisible during startup and that the command

prompt mainly appeared when pressing Escape for troubleshooting. We did not establish GRUB as the cause of the black screen and avoided unnecessary EFI or GRUB reinstallation.

```
grub> ls
grub> normal
```

Part 3: Investigating the Graphics System

5. Identifying graphics hardware

```
lspci -nnk | grep -A 4 -E 'VGA compatible controller|Display controller|3D controller'
```

The Intel UHD Graphics 630 used the i915 driver. The AMD Radeon Pro 5500M used amdgpu. Both GPUs were recognized by Linux. This did not guarantee that graphical startup would succeed, but it showed that drivers were bound to the hardware.

6. Examining kernel logs

```
journalctl --list-boots
sudo journalctl -b -1 -k --no-pager
sudo journalctl -b -4 -k --no-pager
```

The logs included Intel messages such as “Failed to find VBIOS tables (VBT)” and “Cannot find any crtc or sizes”; AMD messages such as “SMU is initialized successfully” and “Display Core initialized”; and workqueue warnings involving i915. Similar warnings appeared on successful boots, so none could be identified as the root cause. The boot index needed careful interpretation: a previous journal boot was not automatically a failed boot.

Part 4: Investigating GNOME and GDM

7. Distinguishing graphical components

GNOME is the desktop environment. GNOME Shell supplies much of the interface and acts as a compositor. Wayland is the display protocol used by the current session. GDM, the GNOME Display Manager, presents the graphical login screen and coordinates session startup. The command below returned wayland:

```
echo $XDG_SESSION_TYPE
# wayland
```

8. Checking Xorg availability

```
ls /usr/share/xsessions/
ls /usr/share/wayland-sessions/
dpkg -l | grep -E 'xserver-xorg-core|gnome-session-xsession|ubuntu-session|wayland'
```

The X11 session directory was missing, while the Wayland sessions directory contained Ubuntu and other session entries. The Xorg server package was installed. These checks helped characterize the display environment but did not isolate the failure.

Part 5: The Breakthrough

9. Examining GDM startup logs

```
journalctl -b -1 --no-pager | grep -Ei 'graphical.target|gdm.service|display
manager|gnome-shell|mutter'
```

The journal showed systemd attempting to start gdm.service. A “Starting GDM” message, however, does not establish that the graphical login screen actually appeared.

10. Starting GDM manually

The decisive test was manually starting GDM after the machine had booted far enough to be usable. The graphical login screen appeared immediately. This demonstrated that the kernel and graphics stack were capable of displaying a login screen. It shifted our attention from “Can GDM work?” to “Does the timing of GDM startup matter?”

Part 6: The Final Hypothesis

11. A graphical startup timing problem

Our working hypothesis was that GDM sometimes started before a graphics-related resource or dependency was ready. A race condition is a failure whose outcome depends on the timing or order of events. If a component attempts to use a resource before it is ready, it may fail; if it starts later, it may succeed. We did not prove which exact resource was involved, so a specific race condition remains a hypothesis rather than a confirmed root cause.

```
Possible failing sequence:
Kernel and GPU initialization begin
    -> systemd starts services
    -> GDM starts too early
    -> black, backlit display

Successful manual sequence:
Kernel and GPU initialization begin
    -> additional time passes
    -> manually start GDM
    -> login screen appears
```

Part 7: The Solution We Implemented

12. Delaying GDM startup

We created a systemd drop-in override rather than editing the packaged GDM unit. Its contents were:

```
[Service]
ExecStartPre=/usr/bin/sleep 30
```

The override was saved at /etc/systemd/system/gdm.service.d/override.conf. The [Service] section modifies service execution. ExecStartPre runs a command before the main GDM startup command. /usr/bin/sleep 30 waits 30 seconds and then exits successfully. It does not stop the entire boot: other system activity can continue during the wait.

13. Why use a systemd override?

The original service file lives at /usr/lib/systemd/system/gdm.service. Editing that packaged file directly could be undone by a package update. Systemd combines the original unit with drop-in configuration. We verified the result and reloaded unit definitions:

```
systemctl cat gdm.service
sudo systemctl daemon-reload
sudo reboot
```

14. Results

The first reboot took approximately 30 extra seconds, then displayed the login screen normally. Several subsequent reboots also succeeded. This is strong evidence that delaying GDM is an effective workaround on the current system. It does not establish which driver, device, or userspace component needs the additional time.

Part 8: Understanding the Entire Linux Boot Process

15. Overall boot sequence

```
POWER BUTTON
    |
    v
1. Apple firmware / EFI / T2 platform
    |
2. GRUB bootloader
    |
3. T2 Linux kernel
    |
4. Initramfs (early userspace)
    |
5. Ubuntu root filesystem
    |
6. systemd (PID 1)
    |
7. System services and device initialization
    |
8. graphical.target
    |
9. GDM display manager
    |
10. Graphical login screen
    |
11. GNOME / Wayland user session
    |
DESKTOP READY
```

This is a conceptual sequence, not a strictly sequential timeline. Device initialization and many services overlap.

Stage 1: Firmware

When powered on, the computer executes firmware before Linux. The firmware prepares the early hardware environment and locates bootable software. On this MacBook Pro, Apple’s T2-based platform is part of that environment. EFI eventually transfers control to a bootloader.

Stage 2: GRUB

GRUB is a launcher. It finds the Linux kernel and initial filesystem image, loads them, and transfers control. Its menu can offer alternate kernels and recovery entries. The grub> prompt is a GRUB command interpreter, not a Linux shell; journalctl and systemctl do not run there.

Stage 3: Linux kernel

The kernel manages CPU scheduling, memory, processes, filesystems, devices, and drivers. The i915 and amdgpu drivers communicate with the graphics processors. The Direct Rendering Manager (DRM) subsystem supplies interfaces for display configuration and graphics integration. We inspected kernel logs because a failure here could prevent later graphical startup.

Stage 4: Initramfs

Initramfs is a temporary in-memory filesystem used during early boot. It includes tools and modules required to find and mount the real root filesystem. It solves early dependencies, such as needing a storage driver before the main filesystem is accessible.

Stage 5: Root filesystem

Linux organizes files under a single root directory, /. Once the Ubuntu root filesystem is mounted, the system can access its installed software, libraries, configuration, and services. Common directories include /boot, /etc, /home, /usr, and /var.

Part 9: Understanding systemd

16. What is systemd?

Systemd is Ubuntu's init system. Its main process normally runs as PID 1 and coordinates the startup and management of services, including NetworkManager, journald, and GDM. It also manages ordering, dependencies, restarts, and targets.

17. Boot is not fully sequential

Systemd starts many independent services concurrently to improve boot performance. This means "a service was started" is not always equivalent to "every function it provides is fully ready." Device and graphics initialization can also continue while userspace services are launching. Timing and readiness therefore matter.

18. Systemd targets

multi-user.target represents a multiuser system with normal non-graphical services. graphical.target represents graphical operation and typically pulls in a display manager. Distinguishing these modes helped separate a Linux boot failure from a graphical-login startup failure.

Part 10: Understanding GDM's Role

19. GDM versus GPU drivers

```
GNOME login screen
       |
GDM / graphical session components
       |
Wayland compositor and graphics libraries
       |
Linux DRM and GPU drivers (i915 / amdgpu)
       |
Physical GPU hardware
```

The kernel drivers interface with the physical GPUs. Userspace components build and present the graphical interface. GDM coordinates the login environment. Manual GDM startup showed these layers could work together once enough time had elapsed.

Part 11: Why the 30-Second Delay Works

20. Before and after

```
BEFORE:                          AFTER:
Kernel starts                    Kernel starts
```

```
Graphics initialize                Graphics initialize
systemd launches services          systemd launches services
GDM starts early                   GDM ExecStartPre runs
Black screen                       Sleep for 30 seconds
                                   GDM starts
                                   Login screen appears
```

The delay does not repair the Intel or AMD drivers, replace firmware, change Wayland, or modify the kernel. It simply changes when the main GDM process starts. If a necessary condition becomes ready during those 30 seconds, GDM can then initialize successfully.

21. Why did systemd not already wait?

Systemd respects explicit ordering and dependency rules. For example, After=some-other.service orders startup relative to that unit, but it does not necessarily prove that every internal operation of that unit has finished. Not every hardware-readiness condition has a matching systemd unit. A fixed delay is a pragmatic workaround; an ideal permanent fix would identify and wait for the precise condition.

Part 12: What We Can and Cannot Conclude

• Confirmed: Ubuntu can boot on the current kernel.

• Confirmed: Both Intel and AMD graphics drivers are bound.

• Confirmed: Manually starting GDM displays the login screen.

• Confirmed: The systemd override was installed and recognized.

• Confirmed: Several reboots succeeded with the 30-second delay.

• Strongly supported: The failure is sensitive to graphical startup timing.

• Plausible, not confirmed: A graphics-related readiness problem is responsible.

• Not established: A specific driver bug or GRUB caused the black screen.

Part 13: Useful Commands to Remember

```
journalctl --list-boots
journalctl -b -1
journalctl -b -1 -k
lspci -nnk
echo $XDG_SESSION_TYPE
systemctl status gdm.service
systemctl cat gdm.service
systemctl get-default
sudo systemctl daemon-reload
```

If Linux boots into a terminal but its graphical interface fails, inspect the display manager and graphics stack before assuming the entire operating system has failed.

Part 14: How to Reverse Our Fix

To edit the delay, open the drop-in with Nano. For example, change sleep 30 to sleep 15, then reload systemd. To remove just this workaround, remove its specific override file and reload systemd. Do not remove it now while it is providing reliable boots.

```
sudo nano /etc/systemd/system/gdm.service.d/override.conf
# Optional change: ExecStartPre=/usr/bin/sleep 15
sudo systemctl daemon-reload

# To remove this specific workaround entirely:
sudo rm /etc/systemd/system/gdm.service.d/override.conf
sudo systemctl daemon-reload
```

Part 15: Bigger Lessons

• A black screen does not necessarily mean Linux failed to boot.

• Hardware driver initialization and graphical startup are distinct stages.

• Timing and ordering can determine whether otherwise working components succeed.

• Log warnings need context and comparison against successful boots.

• Manual service startup is a powerful diagnostic technique.

• Small, reversible changes are often preferable to broad kernel, driver, or EFI modifications.

Final Summary

We began with an intermittent black-screen failure that appeared to implicate the kernel or graphics drivers. The investigation established that GRUB could launch Ubuntu, the T2 Linux kernel and GPU drivers could initialize, and the graphical environment could function. Most importantly, manually starting GDM worked immediately. A systemd drop-in that delays GDM by 30 seconds then produced successful boot after successful boot.

```
[Service]
ExecStartPre=/usr/bin/sleep 30
```

The most defensible conclusion is that graphical startup is sensitive to timing on this configuration. The workaround is effective, while the exact underlying readiness condition remains unproven. Separating boot into its individual stages let us move from “Ubuntu will not boot” to the much more precise observation “GDM does not reliably start at its original startup time.”

Take care, and enjoy having Ubuntu back!